COMP1D

Méabh O'Connor

PROJECT SPECIFICATION

2026

Instructions

You will create the **core functionality** for this project as part of your **coursework** at home/in your 1 hour labs.

In **Week 11** during your **2 hour lab**, you will be required to **extend your program** by adding **three** additional features **under exam conditions**. This exam will be **open book** but you will be **disconnected** from the internet. It is therefore essential that you **fully understand** the code you submit and how your program works.

You are required to use **version control** (e.g. Git) throughout the development of this project. Your repository should show **clear evidence of incremental development** over time, rather than a single large commit.

You should make **regular commits** as you build your solution, for example

- after completing the file reading functionality
- after implementing each menu option
- after testing or fixing bugs.

Each **commit message** should clearly **describe what was added** or changed (e.g. "Added delete function", "Fixed bug in status update").

Your version control history will form **part of the assessment**, with **marks awarded for consistent and regular commits**, clear and meaningful commit messages, and evidence of step-by-step development.

Submitting a repository with only one or very few large commits will result in **reduced** marks, as it does not demonstrate your development process.

N.B**.** While you may choose to use tools such as **AI** to **support** your learning during development, you must **ensure** that you **document** this in comments and **understand any code you include in your project**. Your commit history should reflect your own understanding and progression, and you may be asked to explain your code and development decisions.

Marking Scheme

	Mark
VC (Git) on core functionality	16%
Menu Option 5	28%
Menu Option 6	28%
Menu Option 7	28%

No marks are awarded for the core functionality but **completing** these tasks will **contribute** to the VC (Git) grade and will **prepare you** for the task of **extending** the functionality with new menu options.

US Master's Golf Tournament Management System

You are required to implement a system in Python to track the leaderboard after round 2 in the US Master's Golf tournament.

Your program will read and manipulate player data stored in a file and provide useful functionality for administrators of the leaderboard.

On **Exam Day**, you will **add** extra features.

You may **not use** **dictionaries**, or the **CSV module**. You may **only use** what you have learned in **MTU**.

Data Set Format

This will be stored in CSV format with the following fields:

GolferID (int),
GolferName (str),
ScoreRelativeToPar (int; negatives allowed),
Round (1 to 4),
CutStatus (1=made cut, 0=missed cut).

A **sample dataset** might be:

```
5012,Scottie Scheffler,-4,2,1
5341,Rory McIlroy,-4,2,1
5897,Jon Rahm,3,2,1
5120,Tiger Woods,9,2,0
5201,Brooks Koepka,-1,2,1
5305,Jordan Spieth,0,2,1
5442,Patrick Cantlay,2,2,1
5550,Hideki Matsuyama,-3,2,1
6101,Bryson DeChambeau,-1,2,1
6112,Xander Schauffele,-3,2,1
6123,Collin Morikawa,1,2,1
6134,Viktor Hovland,4,2,0
6145,Matt Fitzpatrick,2,2,1
6156,Shane Lowry,0,2,1
6167,Tommy Fleetwood,-2,2,1
6178,Sepp Straka,5,2,0
```

Core Functionality – Starter Project

Your program will include the following menu of options, offered **repeatedly** as long as the user does not choose Quit:

Option 1: View Leaderboard

Write a function that receives all 5 lists and displays the leaderboard in a formatted table.

- Print **plus sign** + before **Par** scores that are > 0
- Print **minus sign** - before **Par** scores that are < 0
- Print **beaming emoji** 😁 ('\U0001F601') where **Cut Status** = 1
- Print **frowning face emoji** ☹️ ("\U00002639\U0000FE0F") where **Cut Status** = 0

Sample rows could be:

#	Player	Par	Round	Cut Status
5341	Rory McIlroy	-4	2	😁
5012	Scottie Scheffler	-4	2	😁
…				
6178	Sepp Straka	+5	2	😟

Option 2: Delete a record

Write a function that **receives** all **5 lists**.

In the function **ask** the user for the **Golfer ID**.

If the ID is **not found**, display an **appropriate message**.

If the ID is **found**, **delete** all data associated with this **player** record along with an appropriate success **message**.

Option 3: Add a new record

Write a function that **receives** all **5 lists**.

In the function **prompt** for a **Golfer ID**.

Print an appropriate **error message** if the Golfer ID **already exists**.

If the ID **does not exist**, **add** this Golfer ID to appropriate list:

Ask the user for :

- **Player Name**
- **Par** relative score

Set **Round** =2 and **Cut Status** = 1 for other associated lists

Print an appropriate **success message**.

Option 4: Update Cut Status

Write a function that receives **3 lists**: **Golfer ID, Player Name** and **Cut Status**.

Ask the user for a **Golfer ID**.

Show a **message** if ID is **not found**.

Otherwise print a **message** showing the **cut status** for the given player e.g.;

```
Shane Lowry has made the cut. Do you wish to change this(y/n)?
```

Or

```
Shane Lowry has missed the cut. Do you wish to change this(y/n)?
```

If the user replies **y**; **toggle** the **Cut Status** (`1=made cut, 0=missed cut`) for this player.

Print **update** successful **message** if cut status changed.

Option 8

Write a function that receives all 5 lists.

Save the data in the lists back to the file.

Program ends.